

-

Set-B QP.docx

Document

give answers for each question (each question for 100 marks)

CS3691 – Embedded Systems and IoT

Set – B Complete Practical Answers (100 Marks Each)

Based on the uploaded question paper

1. 8051 Assembly Program for 16-bit Addition

Aim

To write an 8051 assembly language program to perform 16-bit addition using direct addressing mode.

Algorithm

1. Load first 16-bit number into registers.
2. Load second 16-bit number.
3. Add lower bytes.
4. Store carry.
5. Add higher bytes with carry.
6. Store result.

Program

```
ORG 0000H
```

```
MOV A,30H
```

```
ADD A,32H
```

```
MOV 40H,A
```

```
MOV A,31H
```

```
ADDC A,33H
```

```
MOV 41H,A
```

```
END
```

Output & Result

- Lower byte result stored in 40H
- Higher byte result stored in 41H

Example:

Address	Value
30H	25H
31H	12H
32H	15H
33H	10H

Result:

- 40H = 3AH
- 41H = 22H

Viva Questions and Answers

1. What is 16-bit addition?

Addition of two 16-bit numbers.

2. Which instruction handles carry?

ADDC instruction.

3. Why use accumulator?

Arithmetic operations occur through accumulator.

4. What is direct addressing?

Memory accessed directly using address.

5. Which flag indicates carry?

Carry flag.

2. 8051 Assembly Program for 16-bit Subtraction

Aim

To perform 16-bit subtraction using direct addressing mode.

Algorithm

1. Load lower bytes.
2. Subtract lower bytes.
3. Store borrow.
4. Subtract higher bytes with borrow.
5. Store result.

Program

ORG 0000H

MOV A,30H

CLR C

SUBB A,32H

MOV 40H,A

MOV A,31H

SUBB A,33H

MOV 41H,A

END

Output & Result

Result stored in 40H and 41H.

Viva Questions

1. Which instruction performs subtraction?

SUBB.

2. Why clear carry before subtraction?

To avoid unwanted borrow.

3. What is borrow?

Carry generated during subtraction.

4. Which register is mandatory?

Accumulator.

5. What is 2's complement?

Method for representing negative numbers.

3. 8-bit Multiplication using 8051

Aim

To multiply two 8-bit numbers.

Algorithm

1. Load first number into A.
2. Load second number into B.
3. Use MUL AB.
4. Store result.

Program

ORG 0000H

MOV A,30H

MOV B,31H

MUL AB

MOV 40H,A

MOV 41H,B

END

Output

- Lower byte in A
- Higher byte in B

Viva Questions

1. Which instruction performs multiplication?

MUL AB.

2. Where is result stored?

A and B registers.

3. Maximum result size?

16 bits.

4. Which registers are used?

A and B.

5. Is carry used?

No.

4. 8-bit Division using 8051

Aim

To divide two 8-bit numbers.

Program

ORG 0000H

MOV A,30H

MOV B,31H

DIV AB

MOV 40H,A

MOV 41H,B

END

Output

- Quotient → A
- Remainder → B

Viva Questions

1. Which instruction divides?

DIV AB.

2. Where is quotient stored?

Accumulator.

3. Where is remainder stored?

B register.

4. What happens if divisor is zero?

Overflow flag set.

5. Is carry affected?

No.

5. Block Transfer Program

Aim

To transfer data block from 4500H to 5500H.

Program

ORG 0000H

MOV R0,#20H

MOV R1,#30H

MOV R2,#05H

```
LOOP:
MOV A,@R0
MOV @R1,A

INC R0
INC R1
DJNZ R2,LOOP

END
```

Viva Questions

1. Which instruction repeats loop?

DJNZ.

2. Why increment pointers?

To access next location.

3. What is block transfer?

Moving multiple bytes.

4. Which registers act as pointers?

R0 and R1.

5. What is indirect addressing?

Memory accessed through registers.

6. Exchange Two Data Blocks

Program

```
ORG 0000H

MOV R0,#20H
MOV R1,#30H
MOV R2,#05H

BACK:
MOV A,@R0
MOV B,@R1

MOV @R0,B
MOV @R1,A

INC R0
INC R1
DJNZ R2,BACK
```

END

Viva Questions

1. What is swapping?

Exchanging values.

2. Which register temporarily stores data?

A and B.

3. Why use loops?

To process multiple bytes.

4. Which instruction decrements and jumps?

DJNZ.

5. What is data block?

Collection of bytes.

7. Find Highest Score

Program

ORG 0000H

MOV R0,#20H

MOV R2,#05H

MOV A,@R0

INC R0

BACK:

CJNE A,@R0,NEXT

NEXT:

JNC SKIP

MOV A,@R0

SKIP:

INC R0

DJNZ R2,BACK

MOV 30H,A

END

Result

Highest value stored at 30H.

Viva Questions

1. Which instruction compares?

CJNE.

2. What does JNC mean?

Jump if no carry.

3. What is sorting?

Arranging data order.

4. Why use accumulator?

Comparison operations.

5. What is maximum value?

Largest data element.

8. Find Least Mark

Program

ORG 0000H

MOV R0,#20H

MOV R2,#05H

MOV A,@R0

INC R0

BACK:

CJNE A,@R0,NEXT

NEXT:

JC SKIP

MOV A,@R0

SKIP:

INC R0

DJNZ R2,BACK

MOV 30H,A

END

Viva Questions

1. What is minimum value?

Smallest element.

2. Which instruction checks carry?

JC.

3. Why use loops?

Repeated comparison.

4. Which register stores result?

Accumulator.

5. What is comparison?

Checking relation between values.

9. Ascending Order Sorting

Program

ORG 0000H

MOV R2,#04H

OUTER:

MOV R0,#20H

MOV R1,#03H

INNER:

MOV A,@R0

INC R0

CJNE A,@R0,CHK

CHK:

JC SKIP

MOV B,@R0

MOV @R0,A

DEC R0

MOV @R0,B

INC R0

SKIP:

DJNZ R1,INNER

DJNZ R2,OUTER

END

Viva Questions

1. What is ascending order?

Smallest to largest.

2. Which sorting method used?

Bubble sort.

3. Why nested loops?

Multiple comparisons.

4. What is swapping?

Exchange values.

5. Time complexity?

$O(n^2)$.

10. Descending Order Sorting

Program

ORG 0000H

MOV R2,#05H

OUTER:

MOV R0,#20H

MOV R1,#04H

INNER:

MOV A,@R0

INC R0

CJNE A,@R0,CHK

CHK:

JNC SKIP

MOV B,@R0

MOV @R0,A

DEC R0

MOV @R0,B

INC R0

SKIP:

DJNZ R1,INNER

DJNZ R2,OUTER

END

Viva Questions

1. What is descending order?

Largest to smallest.

2. Which algorithm used?

Bubble sort.

3. Why compare adjacent values?

For sorting.

4. Which instruction decrements?

DJNZ.

5. What is sorting efficiency?

Speed of sorting.

11. Find Total Marks

Program

ORG 0000H

MOV R0,#20H

MOV R2,#05H

MOV A,#00H

BACK:

ADD A,@R0

INC R0

DJNZ R2,BACK

MOV 30H,A

END

Viva Questions

1. Which instruction adds?

ADD.

2. What is accumulator?

Main arithmetic register.

3. Why use loop?

Repeated addition.

4. Where result stored?

30H.

5. What is overflow?

Result exceeds capacity.

12. Bit Manipulation and Logical Operations

Program

ORG 0000H

MOV A,#55H
ANL A,#0FH
MOV 30H,A

MOV A,#55H
ORL A,#0FH
MOV 31H,A

MOV A,#55H
XRL A,#0FH
MOV 32H,A

CPL A
MOV 33H,A

END

Viva Questions

1. What is AND operation?

Bitwise multiplication.

2. What is OR operation?

Bitwise addition.

3. What is XOR?

Exclusive OR.

4. What does CPL do?

Complement bits.

5. What are logical operations?

Bit manipulation operations.

13. 8051 C Program for Addition

```
#include <reg51.h>

void main()
{
    unsigned char a=10,b=20,c;

    c=a+b;

    P1=c;

    while(1);
}
```

Viva Questions

1. What is unsigned char?

8-bit positive data type.

2. Which header used?

reg51.h.

3. Why infinite loop?

Keeps program running.

4. Which port used?

P1.

5. What is embedded C?

C for microcontrollers.

14. 8051 C Program for Subtraction

```
#include <reg51.h>

void main()
{
    unsigned char a=30,b=10,c;

    c=a-b;

    P1=c;

    while(1);
}
```

15. 8051 C Program for Multiplication

```
#include <reg51.h>
```

```
void main()  
{  
  unsigned char a=5,b=4;  
  unsigned int c;
```

```
  c=a*b;
```

```
  P1=c;
```

```
  while(1);  
}
```

16. 8051 C Program for Division

```
#include <reg51.h>
```

```
void main()  
{  
  unsigned char a=20,b=5,c;
```

```
  c=a/b;
```

```
  P1=c;
```

```
  while(1);  
}
```

17. Send 00–FF to Port P2

```
#include <reg51.h>
```

```
void main()  
{  
  unsigned char i;
```

```
  while(1)  
  {  
    for(i=0;i<=255;i++)  
    {  
      P2=i;  
    }  
  }  
}
```

18. Send ASCII Hex Values to P3

```
#include <reg51.h>

void main()
{
    char arr[]={'0','1','2','3','4','5','A','B','C','D'};
    int i;

    while(1)
    {
        for(i=0;i<10;i++)
        {
            P3=arr[i];
        }
    }
}
```

19. Toggle All Bits of P2

```
#include <reg51.h>

void delay()
{
    int i,j;
    for(i=0;i<300;i++)
        for(j=0;j<1275;j++);
}

void main()
{
    while(1)
    {
        P2=0xFF;
        delay();

        P2=0x00;
        delay();
    }
}
```

20. Send -3 to +3 to Port P3

```
#include <reg51.h>

void main()
{
```

```
signed char i;

while(1)
{
for(i=-3;i<=3;i++)
{
P3=i;
}
}
}
```

21. Toggle P2.1 25,000 Times

```
#include <reg51.h>

sbit LED=P2^1;

void main()
{
unsigned int i;

for(i=0;i<25000;i++)
{
LED=~LED;
}
while(1);
}
```

22. Toggle P4 Continuously

```
#include <reg51.h>

void delay()
{
int i,j;
for(i=0;i<300;i++)
for(j=0;j<1275;j++);
}

void main()
{
while(1)
{
P4=0xFF;
delay();
}
```



```
P4=0x00;
delay();
}
}
```

23. Toggle P2 with 500 ms Delay

```
#include <reg51.h>
```

```
void delay()
{
    int i,j;
    for(i=0;i<500;i++)
        for(j=0;j<1275;j++);
}
```

```
void main()
{
    while(1)
    {
        P2=~P2;
        delay();
    }
}
```

24. Count 00 to FF on LEDs

```
#include <reg51.h>
```

```
void delay()
{
    int i,j;
    for(i=0;i<300;i++)
        for(j=0;j<1275;j++);
}
```

```
void main()
{
    unsigned int i;

    while(1)
    {
        for(i=0;i<=255;i++)
        {
            P3=i;
            delay();
        }
    }
}
```

```
}  
}  
}
```

25. Read from P2 and Send to P3

```
#include <reg51.h>  
  
void delay()  
{  
    int i,j;  
    for(i=0;i<1000;i++)  
        for(j=0;j<1275;j++);  
}  
  
void main()  
{  
    unsigned char data;  
  
    while(1)  
    {  
        data=P2;  
        delay();  
        P3=data;  
    }  
}
```

26. Send Data Depending on Value

```
#include <reg51.h>  
  
void main()  
{  
    unsigned int data;  
  
    while(1)  
    {  
        data=P1;  
  
        if(data>500)  
            P2=data;  
  
        else  
            P3=data;  
    }  
}
```

27. Toggle Only P3.6

```
#include <reg51.h>
```

```
sbit BIT=P3^6;
```

```
void main()
{
while(1)
{
BIT=~BIT;
}
}
```

28. Monitor P2.4

```
#include <reg51.h>
```

```
sbit SW=P2^4;
```

```
void main()
{
while(1)
{
if(SW==1)
P1=0xAA;

else
P1=0x55;
}
}
```

29. Door Sensor and Buzzer

```
#include <reg51.h>
```

```
sbit SENSOR=P2^1;
sbit BUZZER=P2^7;
```

```
void delay()
{
int i,j;
for(i=0;i<100;i++)
for(j=0;j<500;j++);
}
```

```

void main()
{
while(1)
{
if(SENSOR==1)
{
BUZZER=1;
delay();

BUZZER=0;
delay();
}

else
{
BUZZER=0;
}
}
}

```

30. Toggle P0, P1, P2 with sfr

```
#include <reg51.h>
```

```

sfr P0=0x80;
sfr P1=0x90;
sfr P2=0xA0;

```

```

void delay()
{
int i,j;
for(i=0;i<500;i++)
for(j=0;j<1275;j++);
}

```

```

void main()
{
while(1)
{
P0=~P0;
P1=~P1;
P2=~P2;

delay();
}
}

```

Viva Questions

1. What is sfr?

Special Function Register.

2. Why use delay?

Visible output changes.

3. What is port toggling?

Changing port bits repeatedly.

4. Which ports used?

P0, P1, P2.

5. What is infinite loop?

Program runs continuously.